Predict First #1 — Association

Prediction: Python would raise an error after deleting the supplier variable because the products require a Supplier object.

Actual Result: Both products could still access and display supplier information after del supplier. This is because del supplier only removes the variable reference. The Supplier object still exists because both Product objects hold references to it.

Changes Made to Test: No changes were needed because the code worked as intended. My prediction was incorrect because I misunderstood how Python object references work.

Predict First #2 — Aggregation

Prediction: Removing a Product from Inventory would destroy the Product object, causing an error when calling physical.display_information().

Actual Result: The Product could still display its information after being removed from Inventory A. The same Product object could also be added to Inventory B.

Changes Made to Test: No changes were needed because the code worked as intended. My prediction was incorrect because removing a Product from an Inventory does not destroy it while another reference still exists.

Predict First #3 — Composition

Prediction: Adding the same Product to two different Orders would create two separate OrderLine objects, but both OrderLines would reference the same Product object.

Actual Result: Each Order created its own OrderLine object, even though both OrderLines referenced the same Product object. Therefore, there were two separate OrderLine objects but only one shared Product object. This demonstrates composition because each Order creates and manages its own OrderLines.

Changes Made to Test: No code changes should be needed if the test confirms this result, as the Order class already creates OrderLine objects internally.

Predict First #4 — Polymorphism

Prediction: PhysicalProduct would display a message stating that physical shipping is required, while DigitalProduct would display a message stating that digital delivery is used and shipping is not required.

Actual Result: PhysicalProduct displayed "This product requires physical shipping", while DigitalProduct displayed "This product is a digital download and does not require shipping/is digitally delivered". Both products used the same get_fulfilment_info() method but produced different outputs depending on their class. This demonstrates polymorphism.

Changes Made to Test: No code changes should be needed if both subclasses display their respective fulfilment messages correctly. This demonstrates polymorphism because both subclasses respond differently to the same method call.

Liskov Substitution Principle (LSP)

Both PhysicalProduct and DigitalProduct can be used wherever a Product object is expected. For example, both subclasses support display_information(), get_fulfilment_info(), and update_stock().

If DigitalProduct overrides update_stock() to raise an error, it would violate LSP because code expecting normal Product behaviour would fail. I would keep the stock-updating behaviour consistent in both subclasses rather than adding special checks inside Inventory.

Polymorphism and LSP

Polymorphism allows both product subclasses to respond differently to the same get_fulfilment_info() method call. LSP ensures that both subclasses can be used safely through the Product abstraction without breaking the calling code.